Robust Level 3 Autonomous Drone Racing by Gate Search and Time-Optimal Contouring Control

1st Ahmet Arda Hız
Learning Systems and Robotics
Technical University of Munich
Munich, Germany
aarda.hiz@tum.de

2nd Deniz Öztürk
Learning Systems and Robotics
Technical University of Munich
Munich, Germany
deniz.oeztuerk@tum.de

Abstract*—Autonomous drone racing tests a controller at the limit of a vehicle's dynamics. The competition is staged as a ladder of difficulty levels, and we advanced through all of them. Level 3 is the last and hardest, so it is the focus of this report and the capstone of the project, since it needs the racing foundation that the earlier levels build. We study this full Level 3 setting, where the gate positions and orientations are randomized and the true pose of an object is revealed only when the drone comes within 0.7 m of it. For most of the project the environment version we developed against reported only a placeholder layout, with every object at the origin, so a controller could not rely on it, and ours instead finds the gates and then races them without depending on the nominal layout. It flies a fixed spiral sweep that climbs above the obstacles and covers the arena, and each gate is discovered when the drone passes within sensor range in the horizontal plane. The discovered gates are handed to a time-optimal Model Predictive Contouring Control (MPCC) stage that races them, with a gate-aware contouring weight and a dynamics-aware progress anchor for reliability. The method is general: a robust setting (`v24_r10`) finishes 12 of 20 runs on fully randomized Level 3 tracks at 18.8 s, and a faster setting (`v24_r10_fast`) finishes 14 of 20 runs on the graded `final.toml` track at 18.4 s. Late in the competition the environment was changed to report a coarse prior, within about 15 cm of the truth, too coarse to thread a 0.4 m gate opening at speed but enough to plan a route; a no-search variant that trusts this prior and refines it on the 0.7 m reveal flies the graded track in 8.7 s. We describe the design, the price the search pays for generality, and how it grew out of a fast known-track racing core that we flew on the real drone at 9.55 s.

I. Introduction

Autonomous drone racing asks a controller to fly a quadrotor through a set of gates in the shortest time without hitting a gate frame or an obstacle. It is a common benchmark because a fast lap forces the vehicle to work near its actuation limit, where small errors are punished at once [1]. Much of the drone racing work assumes a known track, where every gate position is fixed before the flight and the task is a single time-optimal trajectory [6], [7]. The full Level 3 setting breaks that assumption. The gate positions and orientations are randomized, and the vehicle only sees the true geometry once it is close.

This report comes out of a course project built as a ladder of levels, where each level switches on more disturbance than the one below. We worked through the lower levels first, on a known track and then under inertial and pose randomization, and each result carried into the next, so the top of the ladder reuses a racing core that the lower levels validated. Level 3 is that top. It randomizes the whole layout, which makes it the hardest and most general level and the end product of the project. The graded evaluation runs on a shared configuration, `final.toml`, with a fixed seed, so all groups are compared on the same randomized draw. We treat it as one more Level 3 draw and never read its revealed nominal layout, which is what our design is built to do.

We study the Level 3 setting of the LSY Drone Racing competition. Gate and obstacle positions are drawn at random, and the true pose of an object is reported only when the drone is within 0.7 m of it. For most of the project the environment version we developed against reported only a placeholder at the origin, so a controller that relied on it could not fly the track. It was changed late in the competition to report a coarse nominal, within about 15 cm of the truth, which is too coarse to thread a gate at speed but enough to plan a route (Section V-F). Our controller does not depend on the nominal layout at all. It finds the gates by flying and racing them once they are found.

The controller has two phases. First it searches. It flies a fixed spiral that climbs above the obstacles and sweeps the arena, and a gate is discovered when the drone passes within sensor range of it. Then it races. The discovered gates are handed to a time-optimal Model Predictive Contouring Control (MPCC) stage [2], [4] that treats path progress as a state and traversal speed as an output, so speed is an outcome of the optimization. The contribution of this report is that controller and the design choices that make it robust on any Level 3 layout.

- A blind gate search that finds every gate without the nominal positions, so the same controller works on any randomized layout.
- A gate-aware contouring weight and a dynamics-aware

progress anchor that keep the racing stage reliable once the gates are known.

We evaluate the controller on randomized Level 3 tracks and on the randomized competition track, and we relate it to a fast known-track racing core that we flew on the real Crazyflie at 9.55 s.

II. Related Work

Time-optimal quadrotor flight through a fixed set of gates is well studied. Foehn et al. compute a full time-optimal trajectory that respects the single-rotor thrust limits [6], and minimum-snap trajectories [7] are a common lighter planning choice. These methods give the fastest known laps on a known track, but they assume the gates are known and do not move.

MPCC was introduced for machining [4] and brought to racing on 1:43 cars [5]. Romero et al. applied a time-optimal MPCC to quadrotor flight and showed it reaches the actuator limit in real time [2]. MPCC++ adds a safe tunnel around the reference as hard constraints [3]. We use the time-optimal MPCC formulation for the racing stage.

Learning-based controllers train a policy that maps the observation to an action in one forward pass and have reached champion-level lap times [13], [14]. They are fast but need training and are harder to inspect. The part that is specific to Level 3 is that the gates are not known in advance, so the controller has to discover them before it can race. We handle this with an explicit search stage rather than a learned exploration policy.

III. Problem Description

We use the LSY Drone Racing environment, which builds on the safe-control-gym benchmark [11] with a Crazyflie 2.1 quadrotor [12] as the model. The vehicle has a mass of 43 g and a thrust-to-weight ratio near 1.9. A track holds four gates and four obstacles. A gate opening is 0.4 m wide inside a 0.72 m frame, so the usable gap is about three vehicle widths.

At each step the controller reads an observation

$$o = [\, p,\ v,\ q,\ G,\ O,\ i \,], \tag{1}$$

with position $p \in \mathbb{R}^3$, velocity $v \in \mathbb{R}^3$, orientation quaternion q, the gate poses G, the obstacle positions O and the index i of the next gate. A gate starts at a nominal pose and switches to its true pose once the drone is within the 0.7 m sensor range. The reveal test uses the horizontal distance between the drone and the gate center, so altitude does not matter for detection. In the lab the true poses come from an external motion capture system (Vicon), which tracks reflective markers on the gates and obstacles. The command is sent at 50 Hz. The environment runs in attitude mode, so the output is

$$a = [\, \phi,\ \theta,\ \psi,\ f \,], \tag{2}$$

a roll, pitch and yaw with a collective thrust f. An onboard controller at 500 Hz tracks a.

The challenge is a ladder of difficulty levels that differ by which disturbances are active. Level 0 has no disturbance. The drone and the full track are known, so it measures raw speed. Level 1 randomizes the drone's inertial properties (mass and inertia) and its start pose, while the track stays known. Level 2 also randomizes the gate and obstacle poses within set bounds, revealed within the sensor range, on a layout that is otherwise fixed. Level 3 randomizes the layout itself, so the gate positions and orientations change every run, which forces the controller to plan online in flight. This is the setting we target. The graded evaluation uses a separate configuration, `final.toml`, released in the last lab session. Like Level 3 it randomizes the whole layout, but with a fixed seed, so every group flies the same draw and the lap times compare directly. It keeps the drone and pose disturbances and reveals each gate and obstacle within 0.7 m, with the reported nominal pose already within 15 cm of the truth. We report on it as well, but our controller stays a general Level 3 policy. It does not read the nominal layout of any track, so even when a nominal prior is available it runs the same blind search instead of exploiting it.

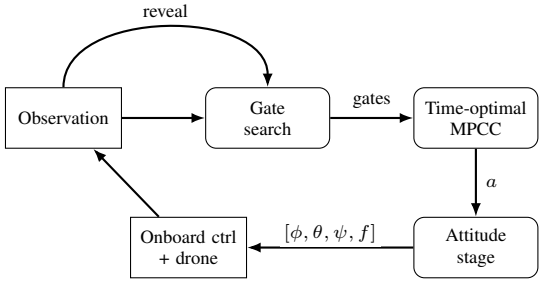

Fig. 1. Controller structure after takeoff. The drone reads the observation, the search discovers the gates, a time-optimal MPCC races them and the attitude stage produces the command. A gate is revealed when the drone passes within 0.7 m of it.

Two features of the setting drive the design. The prior is at best coarse, so the controller cannot fly straight to a gate opening; it must find or refine each gate first. And a gate is seen late, so once the layout is known the racing stage still has to correct for the last few centimeters of a revealed pose within a short window.

IV. Methodology

A. Overview

The controller runs three phases. A short, aggressive launch, which we call the fast launch, climbs off the pad in a fixed brief window instead of a slow hover takeoff, then it enters the search and race loop of Fig. 1. In the search phase it sweeps the arena and discovers the gates. In the race phase it hands the discovered gates to a time-optimal MPCC that flies through them and returns a world-frame acceleration, which an attitude stage turns into a thrust and attitude command. The loop runs at 50 Hz. Nothing in the controller reads the nominal layout, so the same phases run on any Level 3 track.

B. Finding the Gates: Spiral Search

On Level 3 the gate positions and orientations are unknown, so the controller must find the gates before it can race. It does

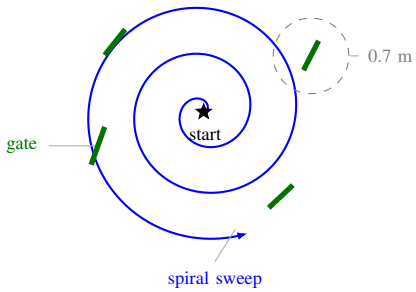

Fig. 2. Blind gate search, seen from above. The drone climbs above the obstacle poles and follows a fixed Archimedean spiral that covers the arena. Each gate (green) is revealed when the spiral passes within 0.7 m of its center in the horizontal plane, drawn as the dashed circle on one gate.

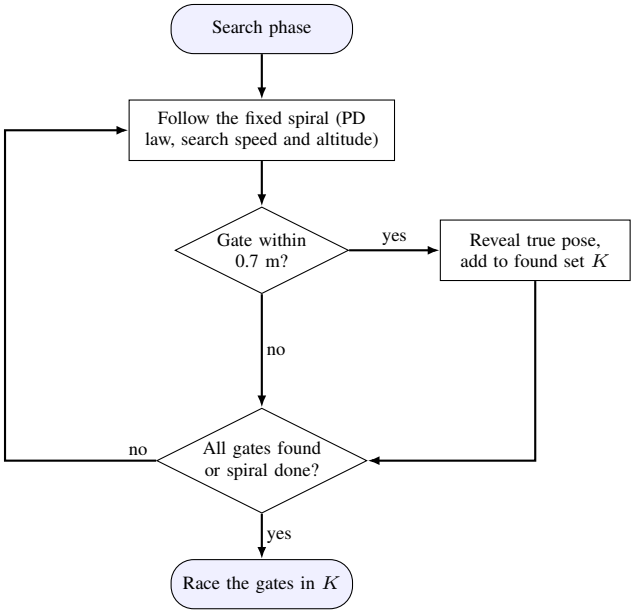

Fig. 3. Gate search, run once per control tick in the search phase. The drone follows the fixed spiral and reveals any gate that comes within 0.7 m in the horizontal plane, adding it to the found set K. It keeps sweeping until every gate is found or the spiral ends, then hands K to the racing stage.

this with a fixed geometric sweep and does not use the nominal positions. After takeoff the drone climbs to a search altitude of 1.8 m, which is above the 1.52 m obstacle poles, so the sweep cannot hit one. It then follows an Archimedean spiral that grows outward from its start position, with a radial step of 0.6 m out to a radius of 2.2 m, clipped to the arena, flown at 1.1 m/s with a PD tracker. Fig. 2 sketches the sweep.

As the drone's horizontal position comes within the 0.7 m sensor range of a gate center, that gate is revealed and its index is collected. The search ends when every gate has been discovered or the spiral is finished, as in Fig. 3. The discovered true poses are then passed to the racing stage. The sweep is blind by design. It covers the arena geometrically, so it works on any layout, and it is the price the controller pays for not knowing where the gates are.

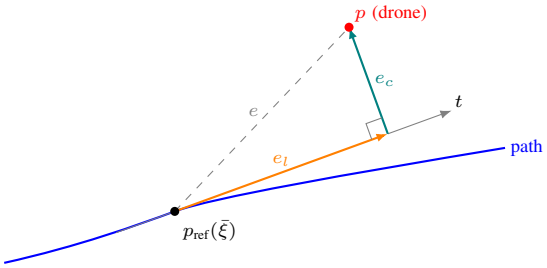

Fig. 4. Contouring geometry. At the reference point $p_{\text{ref}}(\bar{\xi})$ the path has tangent t. The error from the drone position p splits into a lag part e_l along t and a contouring part e_c across it, and the two meet at a right angle at the foot. Together they form the full error e. MPCC penalizes the contouring error hard and the lag softly, and rewards progress so the drone flies as fast as the constraints allow.

C. Time-Optimal MPCC

Once the gates are known the racing stage is a time-optimal MPCC problem [2], [4]. A planner fits a clamped cubic spline through the discovered gates, with two virtual columns per gate at the frame edges so the spline threads the opening center, and a keep-out radius around the obstacles. MPCC adds a progress variable ξ that runs along this path and a progress rate v_ξ. The state is

$$x = [\,p^\top,\ v^\top,\ \xi,\ v_\xi\,]^\top, \tag{3}$$

with control acceleration a and progress acceleration u_ξ and a double-integrator model,

$$\dot{p} = v, \quad \dot{v} = a, \quad \dot{\xi} = v_\xi, \quad \dot{v}_\xi = u_\xi. \tag{4}$$

The reference is linearized around a predicted progress $\bar{\xi}$ with tangent $t(\bar{\xi})$, and the path error splits into a lag part along the tangent and a contouring part across it (Fig. 4),

$$e = p - \big(p_{\text{ref}}(\bar{\xi}) + t(\bar{\xi})(\xi - \bar{\xi})\big), \quad e_l = e^\top t, \quad e_c = e - e_l\, t. \tag{5}$$

The stage cost trades path accuracy against time,

$$\ell = q_c(\xi)\,\|e_c\|^2 + w_l\, e_l^2 - \mu\, v_\xi + w_a\|a\|^2 + r_v\, u_\xi^2. \tag{6}$$

The term $-\mu\, v_\xi$ rewards progress, so the solver flies as fast as the constraints allow, and $q_c(\xi)$ is a per-stage weight (Section IV-D). The actuator limits enter as constraints on the commanded thrust $f = a + g$,

$$\|f\|^2 \le a_{\max}^2, \quad f_z \ge a_{z,\min}, \quad (\kappa_t f_z)^2 \ge f_x^2 + f_y^2, \tag{7}$$

the thrust magnitude, a minimum lift and a tilt limit. A friction-circle curvature cap completes the set,

$$v_{\text{curv}}(\xi) = \text{clip}\Big(\sqrt{a_{\text{lat}}/\kappa(\xi)},\ v_{\min},\ v_{\max}\Big), \tag{8}$$

which soft-bounds the speed and the progress rate. We solve the program in real time with acados [8] using a sequential quadratic programming real-time iteration [10], with the quadratic programs handled by HPIPM [9]. The horizon is 18 steps at 0.05 s, and the first acceleration of the horizon is the command.

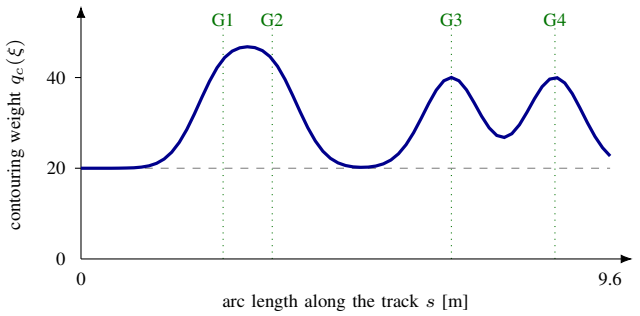

Fig. 5. The gate-aware contouring weight along the track, computed from (9) for the competition layout. The weight sits at a low base on the straights and rises to a peak at each gate (G1 to G4), so tracking is stiff only where a gate must be threaded. Gates whose arc positions fall within the window merge into a single wider peak.

D. Gate-Aware Contouring Weight

A single contouring weight is wrong somewhere on the track. High everywhere brakes the vehicle on the straights, and low everywhere lets it drift at a gate, where a drift is a miss. We make the weight a function of progress,

$$q_c(\xi) = q_{\text{base}} + q_{\text{gate}} \sum_j \exp\left(-\frac{(s(\xi) - s_j)^2}{2\sigma^2}\right), \qquad (9)$$

with s_j the arc position of gate j and σ the window width. The weight is low between gates and peaks at each gate, shown in Fig. 5. The vehicle is held on the line where a gate must be threaded and runs free elsewhere, which lets the speed rise without a drop in the gate pass rate.

E. Dynamics-Aware Progress Anchor

The MPCC needs a value of the progress ξ each step. The direct choice, the arc length of the nearest path point, fails at a hairpin, where the path folds back and a far leg comes close in space, so the nearest-point search jumps to it and skips a gate. We anchor the progress to the solver's own predicted progress one step ahead, which advances at the bounded rate v_ξ and cannot jump, and correct it with a nearest-point search limited to a band around the prediction. The far leg of a fold lies outside the band and is never selected, while the gate motion lies inside it. The anchor removes the fold skip without changing tracking elsewhere.

F. Two Settings

We use two settings of the same controller, labeled by their configuration names. Both run the same blind search and the same MPCC, and neither reads the nominal layout, so the difference is aggressiveness and not method. The `v24_r10` setting keeps a moderate speed budget so that it finishes on as many random draws as possible, and it leads on fully randomized Level 3. The `v24_r10_fast` setting is more aggressive: it raises the MPCC speed budget and tightens the search, so it reaches the gates sooner and races them harder. On randomized Level 3 it cuts the lap time by about a third, from 18.8 s to 12.8 s, at a small cost in success rate. On the larger `final.toml` arena `v24_r10` is the most reliable setting, but its slow spiral makes the lap long, while `v24_r10_fast` covers the arena sooner and still finishes well over half its draws. Lap time is the racing objective and both settings clear a comfortable success margin, so `v24_r10_fast` is our choice for the graded track. The two settings differ only in these parameters, so this is an aggressiveness choice and not a change of method. Table II reports both.

V. Evaluation and Results

A. Setup

We evaluate the controller on randomized Level 3 tracks with the Level 3 evaluation script, and on the randomized competition configuration (`final.toml`) with the provided seed. Fig. 6 shows a lap on a `final.toml` draw. Both settings use at least 20 runs. The controller reads no nominal layout in either case, and the search parameters and the MPCC weights are shared across all tracks. Table I lists the main parameters.

TABLE I
Main Parameters

Parameter	Value
Search altitude	1.8 m
Search speed	1.1 m/s
Spiral step / max radius	0.6 / 2.2 m
Sensor range (horizontal)	0.7 m
Obstacle keep-out	0.10 m
MPCC horizon / step	18 / 0.05 s
Race top speed $v_{\max}$	3.2 m/s
Race lateral budget a_{lat}	8.5 m/s^2
Control rate	50 Hz

B. Main Result

Table II reports the controller across settings and tracks. Each run draws a fresh random layout, so both the `level3.toml` and `final.toml` columns measure generalization across 20 distinct layouts, not one layout under disturbance. On fully randomized Level 3 the `v24_r10` setting finishes 12 of 20 runs at 18.8 s, and `v24_r10_fast` cuts the lap time to 12.8 s at 10 of 20, trading about ten points of success for a third off the lap time. On the graded `final.toml` draws `v24_r10` is the more reliable setting at 17 of 20, but its slow sweep makes the lap long at 26.9 s, while `v24_r10_fast` finishes 14 of 20 at 18.4 s. Both clear a comfortable margin above half the draws, and lap time is the racing objective, so we promote the faster `v24_r10_fast` as our headline result on the graded track: it is about a third quicker and still finishes well over half its draws. The remaining misses on either track come from draws that place gates and obstacles in tight or crossing configurations that the racing stage cannot thread.

C. The Price of Search

The lap time is dominated by the search, not by the racing. The spiral has to cover the arena before every gate is seen,

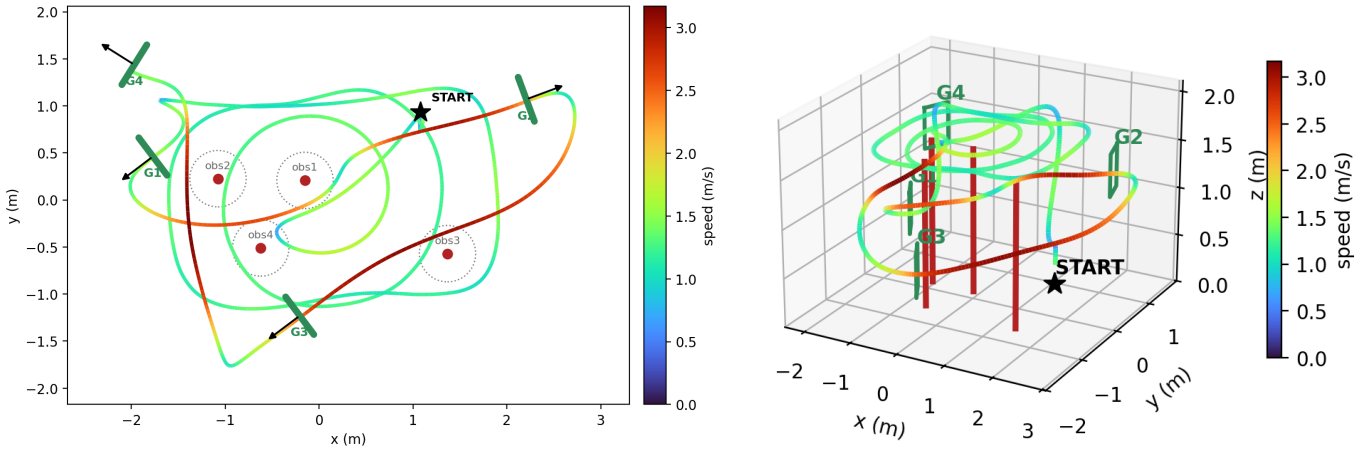

Fig. 6. A run on a randomized `final.toml` draw, from the simulator, colored by speed. Left, top-down: the wide loops are the blind spiral search that covers the arena; once all four gates are found the drone races them, slow (blue to green) at the gates and tight turns and fast (orange to red) on the straights. Green bars are the gate openings with the crossing direction, red marks are obstacles with their keep-out and the star is the start. Right, three-dimensional: the search spiral runs high, above the obstacle poles, then the drone descends to race the gates.

TABLE II
THE CONTROLLER ACROSS SETTINGS AND TRACKS. ALL TRACKS ARE RANDOMIZED; EACH ROW IS AT LEAST 20 RUNS OVER DISTINCT LAYOUTS. THE LAST ROW IS THE NO-SEARCH VARIANT.

Setting	Track	Success	Lap (s)
`v24_r10`	`level3.toml`	12/20	18.83
`v24_r10_fast`	`level3.toml`	10/20	12.81
`v24_r10`	`final.toml`	17/20	26.91
`v24_r10_fast`	`final.toml`	14/20	18.36
`v24_r10_nosearch`	`final.toml`	11/20	8.74

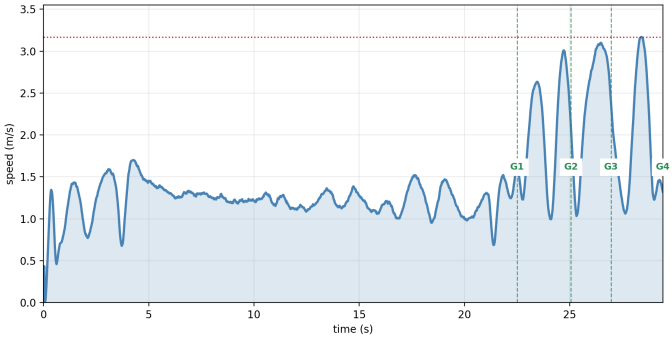

Fig. 7. Speed against time on a `final.toml` draw. The long low-speed section is the blind spiral search covering the arena, and the short high-speed section is the race through the four discovered gates. The search, not the racing, sets the lap time.

and this takes on the order of 20 s on `final.toml`. The racing itself, once the gates are known, takes only a few seconds. Fig. 7 shows this split in the speed trace: a long low-speed sweep while the arena is covered, then a short fast burst through the four gates. This is why the search settings reach 18 to 27 s on a track that a known-track racer flies in under 10 s. The gain in return is generality. Because the search never reads the nominal layout, the same controller runs on any Level 3 draw, which is what the setting asks for.

D. Racing-Core Robustness

Table III isolates the three components we add to the racing core, on a development track with the gates known and no search stage, over 60 runs. This is why the laps here are near 8 s and not the 18 to 26 s of the full runs. The gate-aware contouring weight lets the raised speed hold the gates. The fast launch cuts the start phase. The dynamics-aware progress anchor costs a little time on this track, but it removes the fold skip on sharp-slalom draws that the plain nearest-point projection cannot handle, and this development track does not exercise those draws. Each component targets a distinct failure and composes with the others.

TABLE III
RACING-CORE COMPONENT EFFECT ON A DEVELOPMENT TRACK (60 RUNS)

Configuration	Success	Mean lap (s)
Gate-aware MPCC	54/60	8.20
+ fast launch	52/60	8.01
+ progress anchor	54/60	8.13

E. Why It Works

The controller separates two problems. Finding the gates is handled by a blind sweep that does not depend on the layout, so it never fails because the nominal positions were wrong. Racing the gates is handled by an optimizer that makes speed an output, with precision spent only where a gate is threaded. Because neither part reads the nominal layout, the same policy runs on any Level 3 track, which is the property the setting is testing.

F. A No-Search Variant

For most of the project the environment version we developed against reported a placeholder nominal layout, with every gate and obstacle at the origin, so a controller could not rely on the nominal and a blind search was our way to fly the track. This is the regime our controller is built for, and it is also the safe assumption for real deployment, where the prior can be wrong or the motion-capture feed can drop out. Late in the competition, after we resynced our fork, the environment was changed to report a coarse nominal pose, within about 15 cm of the truth and still refined within 0.7 m. This prior is too coarse to thread a 0.4 m gate opening at speed but good enough to plan a route, so we added a `v24_r10_nosearch` variant that trusts it: it skips the spiral and hands the reported gates straight to the racing core, refining each pose on reveal. On `final.toml` it finishes 11 of 20 draws at 8.7 s (Table II), a threefold cut in lap time over the `v24_r10` search setting. The two ends form one controller that degrades gracefully with the quality of the prior: a full blind search when there is no usable prior, direct racing when the prior is good, and the same racing core in between. We submit the search setting because it holds under the weakest prior and, even where the coarse prior is available, is the more reliable choice: `v24_r10_fast` finishes 14 of 20 against the no-search variant's 11 of 20. We report `v24_r10_nosearch` as the fast path available when a usable prior is present.

VI. Development Process

The controller is the result of several iterations, and the search stage was the last piece. We first built a global planner that fits a spline through the gates and a time-optimal MPCC that races it. On a known track this races fast. We pushed an aggressive configuration of the racing core to the actuator limit, raising the progress reward, the top speed, the lateral budget and the tilt limit. We flew that aggressive configuration on the real Crazyflie and completed the physical lab track in 9.55 s, which validated the racing core on hardware.

That fast configuration, however, reads the gate positions and so only works when the layout is known. The open problem was the full Level 3 setting, where the layout is random. We therefore added the blind gate search in front of the same racing core. The search discovers the gates without the nominal positions, and the racing core then flies them. The submitted controller pays a fixed search cost on every track, including the graded one, in exchange for working on any Level 3 layout. We keep two settings of it, `v24_r10` and the more aggressive `v24_r10_fast`, which is the same progression from safe to fast that we followed on the racing core. The aggressive real flight and the general search-based controller share the same racing engine, so the hardware result also validates the core of the submitted controller.

VII. Limitations and Future Work

The main cost is the blind search. Even on a track whose layout is close to nominal, the controller flies the full sweep, which dominates the lap time. The no-search variant of Section V-F already removes most of that cost when a nominal prior is available; extending it to a hybrid that starts from the nominal positions and searches only where a gate is not found would keep the blind sweep as a fallback for the placeholder case. The search altitude and speed trade coverage against time, and a tighter or faster sweep would find the gates sooner. On randomized tracks the misses come from draws that place gates and obstacles too close to thread, which is a property of the layout rather than the controller. Future work is an adaptive search that uses a nominal prior when one is available, and a reveal-aware racing law that shrinks the last-meter correction at a gate.

VIII. Conclusion

We presented a general Level 3 controller for autonomous drone racing under randomized layouts. It finds the gates with a blind spiral sweep that does not use the nominal positions, then races them with a time-optimal MPCC that makes speed an output. A gate-aware contouring weight and a dynamics-aware progress anchor keep the racing stage reliable. The `v24_r10` setting finishes 12 of 20 runs on fully randomized Level 3 at 18.8 s, and the faster `v24_r10_fast` setting finishes 14 of 20 runs on the graded `final.toml` track at 18.4 s. Its racing core is the same one we flew on the real drone at 9.55 s in a fast known-track configuration. The design trades lap time for generality, which is the trade the full Level 3 setting demands.

Appendix A
Reproducibility

The controller is evaluated on randomized Level 3 tracks and on the randomized `final.toml` competition track with its fixed seed, in the LSY Drone Racing environment, in attitude control mode at 50 Hz, with the Crazyflie 2.1 model, and requires the acados solver toolchain. The search parameters, the planner and the MPCC weights are shared across all tracks, and the submitted search controller uses no nominal layout. The controller code is available at https://github.com/denizozturk95/lsy_drone_racing.

References

[1] D. Hanover, A. Loquercio, L. Bauersfeld, A. Romero, R. Penicka, Y. Song, G. Cioffi, E. Kaufmann, and D. Scaramuzza, "Autonomous drone racing: A survey," IEEE Trans. Robot., vol. 40, pp. 3044–3067, 2024.

[2] A. Romero, S. Sun, P. Foehn, and D. Scaramuzza, "Model predictive contouring control for time-optimal quadrotor flight," IEEE Trans. Robot., vol. 38, no. 6, pp. 3340–3356, 2022.

[3] M. Krinner, A. Romero, L. Bauersfeld, M. Zeilinger, S. Coros, and D. Scaramuzza, "MPCC++: Model predictive contouring control for time-optimal flight with safety constraints," in Robotics: Science and Systems (RSS), 2024.

[4] D. Lam, C. Manzie, and M. Good, "Model predictive contouring control," in IEEE Conf. Decision and Control (CDC), 2010, pp. 6137–6142.

[5] A. Liniger, A. Domahidi, and M. Morari, "Optimization-based autonomous racing of 1:43 scale RC cars," Optim. Control Appl. Methods, vol. 36, no. 5, pp. 628–647, 2015.

[6] P. Foehn, A. Romero, and D. Scaramuzza, "Time-optimal planning for quadrotor waypoint flight," Sci. Robot., vol. 6, no. 56, 2021.

[7] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in IEEE Int. Conf. Robotics and Automation (ICRA), 2011, pp. 2520–2525.
[8] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, and M. Diehl, "acados: a modular open-source framework for fast embedded optimal control," Math. Program. Comput., vol. 14, pp. 147–183, 2022.
[9] G. Frison and M. Diehl, "HPIPM: a high-performance quadratic programming framework for model predictive control," in IFAC World Congress, 2020, pp. 6563–6569.
[10] M. Diehl, H. G. Bock, and J. P. Schlöder, "A real-time iteration scheme for nonlinear optimization in optimal feedback control," SIAM J. Control Optim., vol. 43, no. 5, pp. 1714–1736, 2005.
[11] Z. Yuan, A. W. Hall, S. Zhou, L. Brunke, M. Greeff, J. Panerati, and A. P. Schoellig, "Safe-control-gym: a unified benchmark suite for safe learning-based control and reinforcement learning in robotics," IEEE Robot. Autom. Lett., vol. 7, no. 4, pp. 11142–11149, 2022.
[12] W. Giernacki, M. Skwierczyński, W. Witwicki, P. Wroński, and P. Kozierski, "Crazyflie 2.0 quadrotor as a platform for research and education in robotics and control engineering," in Int. Conf. Methods and Models in Automation and Robotics (MMAR), 2017, pp. 37–42.
[13] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," Nature, vol. 620, pp. 982–987, 2023.
[14] Y. Song, M. Steinweg, E. Kaufmann, and D. Scaramuzza, "Autonomous drone racing with deep reinforcement learning," in IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS), 2021, pp. 1205–1212.